metroLog — система учёта средств измерений и эталонов

Клиент-серверная архитектура: React SPA → nginx → FastAPI → PostgreSQL / Redis

Дисциплина: «Разработка клиент-серверных приложений»

Студент: Макеев Глеб Борисович, группа BHEBDPS-24-RKSP-5

Преподаватель: Бучинский И. Н.

18.10.2026 · github.com/mflkee/metroLog · v0.13.0

Проблема и назначение системы

Как было (AS IS)

- реестр СИ/ЭСИ — в разрозненных Excel-файлах по папкам и подразделениям;
- сроки поверки контролируются вручную по календарю — риск просрочки;
- данные из ФГИС «Аршин» переносятся руками: опечатки и дубли;
- статус прибора нигде не агрегирован; нет разграничения доступа к файлам.

Назначение и результат

- единая база измерительного оборудования с иерархией «папка → группа → прибор»;
- автоматический контроль сроков и синхронизация с ФГИС «Аршин»;
- прозрачная история поверок и ремонтов; доступ заказчика — только к своим папкам;
- **результат:** работающая система v0.13.0: 91 путь API, 124 операции, 18 экранов, 196 backend-тестов.

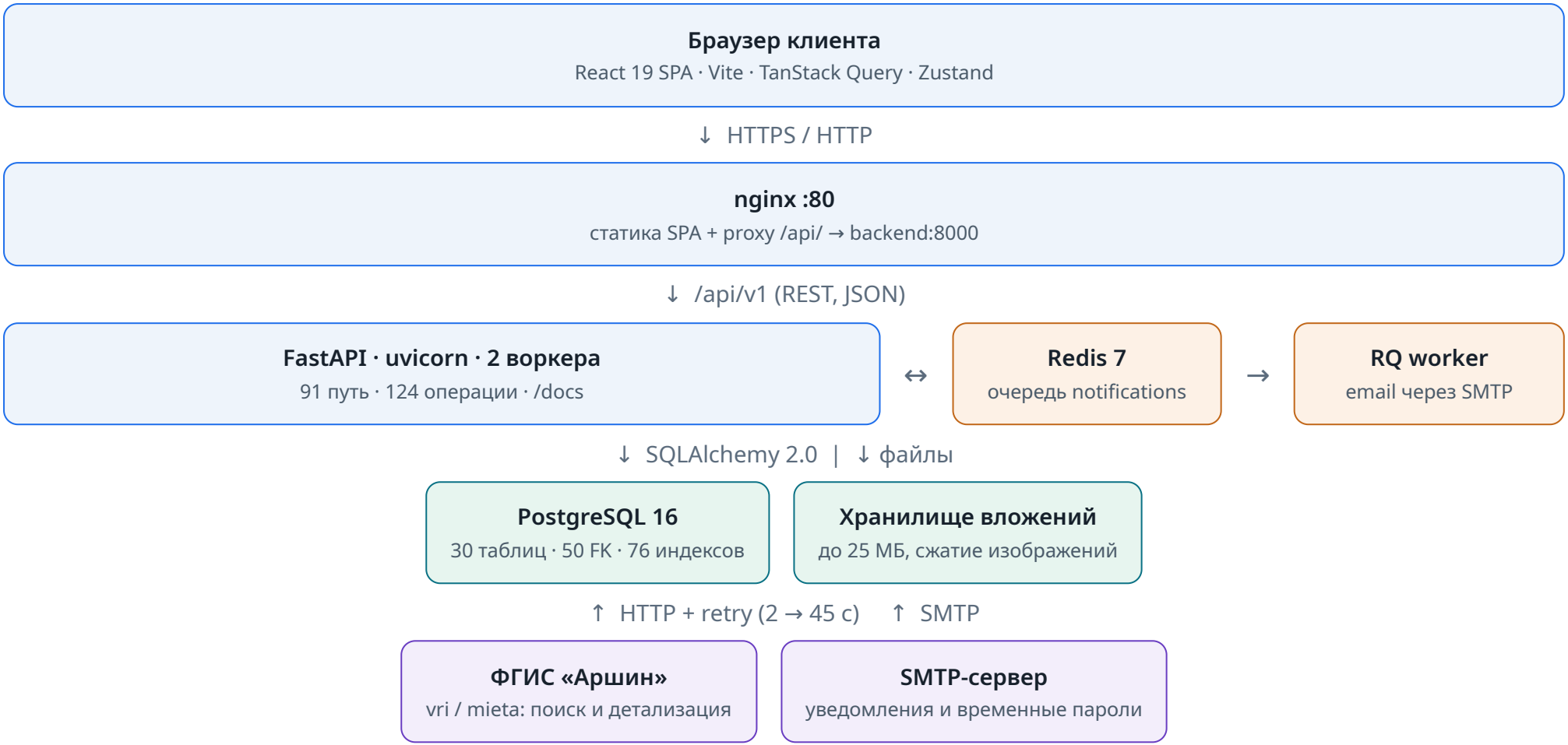
Пользователи и роли

Роль	Код	Что может
Разработчик	DEVELOPER	поддержка и диагностика, полный доступ
Администратор	ADMINISTRATOR	пользователи, роли, права по папкам, справочники
Сотрудник метрологической службы	MKAIR	реестр, поверки, ремонты, переписка, отчёты
Заказчик	CUSTOMER	просмотр своих папок, комментарии, согласование

Два уровня разграничения

- **роль** — какой класс операций разрешён (чтение / оператор / администратор);
- **скоуп папок** — белый список разрешённых и чёрный список скрытых папок;
- нет доступа к объекту → **404**, а не 403: система не раскрывает сам факт существования чужих данных.

Диаграмма развёртывания



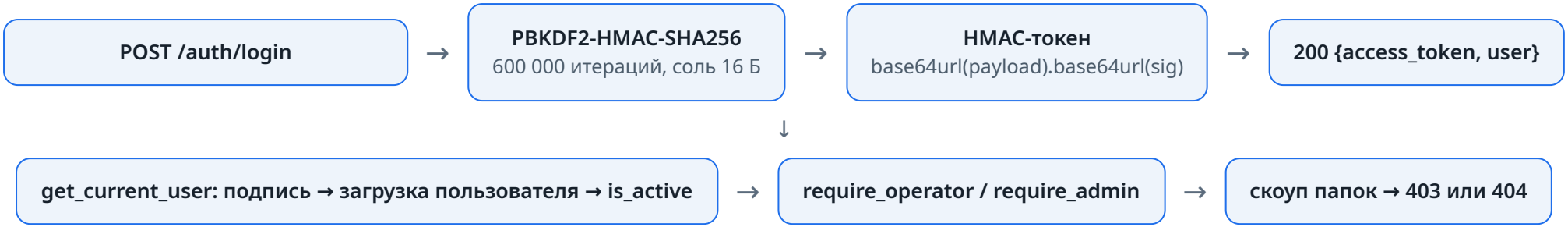
Один хост, пять контейнеров Docker Compose. Наружу открыт только nginx: раздаёт статику и проксирует /api/ на backend. PostgreSQL, Redis и воркер доступны лишь во внутренней docker-сети (порты БД и Redis — только на loopback).

Компоненты и ответственность

Слой	Технология	Зона ответственности
Клиент	React 19 + TS, Vite, TanStack Query, Zustand	экраны, кэш серверных данных, состояние сессии и темы
Шлюз	nginx 1.27	статика SPA, try_files, proxy /api/, без буферизации загрузок
API	FastAPI + Pydantic	маршрутизация, валидация, OpenAPI, зависимости
Логика	сервисный слой (18 модулей)	скоуп папок, процессы, batch-операции, уведомления
Данные	SQLAlchemy 2.0 + Alembic	ORM, 66 миграций, транзакции, репозитории
Фон	Redis 7 + RQ	email-уведомления, повторы, fallback в синхронную отправку
БД	PostgreSQL 16	30 таблиц, 50 FK, 76 индексов (8 частичных)

> Ключевое решение: бизнес-логика не в эндпоинтах, а в сервисах — роутер отвечает только за HTTP-контракт, поэтому правила доступа и процессов тестируются отдельно от транспорта.

Поток данных: вход в систему (HMAC, stateless)



1. Клиент: POST /api/v1/auth/login {email, password}.
2. Pydantic-валидация → AuthService.login(); пароль проверяется PBKDF2-HMAC-SHA256, 600 000 итераций.
3. Сервер формирует токен: base64url(payload) . base64url(HMAC-SHA256(payload, secret)), payload = {sub, role, iat, exp}.
4. Ответ 200 {access_token, user}; клиент подставляет Authorization: Bearer <token>.
5. Сервер **не хранит сессии**: отзыв = блокировка пользователя (проверка на каждом запросе) или ротация ключа.

Поток данных: CRUD оборудования

Операция	Запрос	Что происходит на сервере
Список	GET /equipment/page?limit=&offset=&search=&sort=	проверка скоупа → SQL с пагинацией → EquipmentRead
Чтение	GET /equipment/{id}	доступ к папке → 404 при отказе → карточка с деталями
Создание	POST /equipment	валидация → вставка → событие аудита → 201
Изменение	PATCH /equipment/{id}	частичное обновление (exclude_unset) → событие аудита
Удаление	DELETE /equipment/{id}	каскад процессов и вложений → событие → 204
Массово	POST /equipment/delete-batch	удаление пачки, событие на каждую сущность

- все запросы к реестру проходят через фильтр скоупа — на уровне сервиса, а не интерфейса;
- списки всегда постраничные; тяжёлые выгрузки — в XLSX;
- каждое изменение оставляет запись в журнале событий с текстовыми копиями названий.

Поток данных: импорт из Аршина и поверка

Сверка с ФГИС «Аршин»

1. POST /arshin/search — поиск по свидетельству, регистрационному номеру, типу прибора.
2. ArshinClient: таймаут 30 с, до 5 попыток, экспоненциальная задержка 2 → 45 с.
3. GET /arshin/vri/{vri_id} — детализация; ответ кэшируется в si_verifications вместе с raw_payload_json.
4. POST /equipment/folders/{id}/refresh-tasks — шапка задачи + строка на каждый прибор: UPDATED / UPDATED_UNCERTAIN / UNCHANGED / NOT_FOUND / ERROR.
5. Неоднозначный заводской номер → serial_ambiguous + список кандидатов; применяется выбранная оператором запись.

Поверка и ремонт

- POST /equipment/{id}/verification — режим (выездная/на месте, с демонтажом или без), маршрут, сроки;
- этапы, переписка и вложения; закрытие — POST …/verification/close;
- групповые операции: POST /equipment/verifications/bulk + PATCH …/batch/{batch_key}.

Доступ: RBAC + скоуп папок

Уровень	Где проверяется	Результат при отказе
Аутентификация	get_current_user (подпись токена, is_active)	401
Роль	require_operator / require_admin	403
Скоуп папок	is_folder_access_allowed в сервисах	**404** (soft-denial)
Ролевые экраны	RequireRoles в React Router	редирект в интерфейсе

- DEVELOPER и ADMINISTRATOR видят всё — скоуп не ограничен;
- MKAIR и CUSTOMER — только allowed_folder_ids минус hidden_equipment_folder_ids;
- проверка применяется к спискам, одиночным объектам и вложенным сущностям (поверки, ремонты, комментарии, задачи);
- почему 404, а не 403: 403 подтверждает существование объекта, а сам факт наличия чужого прибора — коммерческая информация.

API-контракт: REST, OpenAPI, slumber

Метод	Пример	Смысл
POST	/api/v1/auth/login	вход, выдача токена
GET	/api/v1/equipment/page?limit=50&offset=0	постраничный список с фильтрами
GET	/api/v1/equipment/{equipment_id}	карточка прибора
POST	/api/v1/equipment	создание (201)
PATCH	/api/v1/equipment/{equipment_id}	частичное изменение
DELETE	/api/v1/equipment/{equipment_id}	удаление (204)
PUT	/api/v1/equipment/folders/{id}/process-subscriptions	идемпотентная замена подписок
POST	/api/v1/equipment/verifications/bulk	групповая операция

- коды: 200/201/204 — успех, 400 — бизнес-ошибка, 401 — нет токена, 403 — нет прав, 404 — нет объекта или доступа, 503 — недоступна БД/Redis;
- OpenAPI 3 генерируется автоматически: /docs (Swagger UI), /redoc, /api/v1/openapi.json — 91 путь, 124 операции;
- slumber.yml — коллекция запросов к ФГИС «Аршин» (vri/mieta: поиск, детализация).

Фон, миграции, эксплуатация

Фоновые задачи (Redis + RQ)

- очередь notifications, воркер отдельным контейнером: python -m app.tasks.worker;
- письма-упоминания, события процессов, уведомления подписчикам, временные пароли;
- таймаут задачи 120 с, TTL результата 300 с, TTL ошибки 24 ч;
- **fallback:** при недоступности Redis письмо уходит синхронно — уведомление не теряется;
- напоминания по срокам идемпотентны: UNIQUE (task_id, reminder_date);
- пакетное обновление папки — BackgroundTasks с прогрессом в БД.

Миграции и эксплуатация

- Alembic: 66 ревизий, alembic upgrade head при старте контейнера backend;
- GET /api/v1/health — живость; GET /api/v1/health/ready — БД (SELECT 1) и Redis, иначе 503;
- контейнеры restart: unless-stopped, перед деплоем — pg_dump базы.

Тестирование и нагрузка

Уровень	Инструмент	Объём
Backend: API и сервисы	pytest + httpx ASGI (SQLite in-memory)	196 тестов в 10 файлах
Frontend: логика	vitest + Testing Library	33 файла тестов
Линт и типы	ruff, mypy, eslint, tsc --noEmit	на каждый push в CI

Нагрузка (Locust, read-профиль)	30 пользователей	60 пользователей
Запросов / RPS	2 311 / 19,3	3 255 / 27,3
Ошибки	0	0
p50 / p95	53 мс / 1,4 с	660 мс / 2,4 с
Максимум	6,1 с	25,5 с

> Узкое место найдено: POST /auth/login — PBKDF2 600 000 итераций считается в event loop и блокирует воркер. Для заявленного профиля (десятки пользователей) запаса хватает, но хеширование нужно вынести в пул потоков (anyio.to_thread.run_sync).

CI/CD

Стадия	Что делает
checks	ruff, pytest с покрытием, eslint, vitest, статический guard UI
version-guard	версия в frontend/package.json и корневом package.json обязана совпадать
build-images	сборка backend и frontend, публикация в ghcr.io/mflkee/metrolog-* (sha-*, staging/latest)
runner-preflight	ждёт self-hosted runner mkair-runner, иначе падает с понятным сообщением
deploy-staging	docker compose pull && up -d на порту 9000 + проверка /health/ready
deploy-prod	pg_dump → pull/up → health-проверка → удаление release-ветки

- **main** → проверки → сборка → staging; **release/X.Y.Z** → тот же конвейер → production;
- сломанный тест или разошедшаяся версия не попадают на прод;
- перед каждым прод-деплоем автоматически снимается резервная копия БД.

Что осталось и выводы

Сделано

- единая БД: 30 таблиц, 3НФ с документированными денормализациями;
- REST API: 91 путь, 124 операции, OpenAPI и Swagger; веб-клиент: 18 экранов;
- интеграция с ФГИС «Аршин» с ретраями и пакетным обновлением;
- 196 pytest + 33 vitest, нагрузочные тесты Locust, CI/CD с деплоем.

Осталось

1. второй протокол взаимодействия (кроме HTTP/REST) — уточнить зачёт SMTP, иначе WebSocket/SSE;
2. фикс PBKDF2: вынести хеширование в пул потоков и перепрогнать нагрузочный тест;
3. тесты на устойчивость: 4xx/5xx, некорректные токены, недоступность Аршина;
4. демонстрация по сценарию приёмки из ТЗ (раздел 11.6).

> Вывод: спроектированная клиент-серверная архитектура закрывает требования практики 1, подтверждена тестами и нагрузочными прогонами и работает в реальной эксплуатации.